

부분 관측 Fully Decentralized MARL에서 동적 방향성 정보 공유 구조

Trajectory 기반 selective critic sharing 연구 구상

이환률

Hanyang University

Contents

- ① 연구 주제 정리
- ② 구체적인 문제 설정
- ③ 방법 구상
- ④ 실험 계획
- ⑤ 예상되는 어려움
- ⑥ 정리

생각하고 있는 주제

부분 관측 fully decentralized MARL에서, 각 agent가 주변 agent의 최근 행동 궤적을 보고 critic 정보를 선택적으로 공유하는 방법을 연구한다.

- 각 agent는 중앙 critic이나 global state를 사용하지 않는다.
- agent는 자기 local observation과 주변에서 보이는 agent의 움직임만 사용한다.
- 고정된 communication graph를 쓰기보다, 학습 중에 참고할 이웃을 계속 바꾼다.
- 공유 대상은 policy 자체보다 critic 또는 critic update 정보에 둔다.

문제를 이렇게 보는 이유

- Fully decentralized actor-critic에서는 critic을 이웃과 섞으면서 global한 정보를 간접적으로 반영한다.
- 그런데 이웃이라고 해서 항상 유용한 정보라고 보기는 어렵다.
- 특히 agent들의 reward, 역할, 관측 범위가 다르면 full sharing은 학습을 흐릴 수 있다.
- 반대로 완전히 독립적으로 학습하면 주변 agent가 가진 정보를 전혀 쓰지 못한다.

Independent learning과 full sharing 사이의 중간 구조가 필요함

기존 graph 기반 decentralized AC와의 차이

- 기존 fully decentralized actor-critic에서는 보통 통신 graph가 미리 정해져 있다.
- 예를 들어 격자 환경에서 가까운 agent, 네트워크상 neighbor 등을 그대로 사용한다.
- 이 경우 graph는 환경 구조를 반영할 수는 있지만, 학습 중 정보의 유용성까지 반영하지는 못한다.

내가 바꾸고 싶은 부분

“가까이 있는 agent”가 아니라, “현재 내 critic update에 참고할 만한 agent”를 이웃으로 보고 싶다.

기존 selective sharing과의 차이

- Selective parameter sharing은 비슷한 agent끼리 policy parameter를 공유하는 방향에 가깝다.
- 내가 생각하는 주제는 agent를 고정된 group으로 나누는 것이 아니다.
- 같은 agent라도 상황에 따라 참고할 수도 있고, 참고하지 않을 수도 있다.
- 따라서 relation은 고정 group보다 시간에 따라 바뀌는 weighted graph에 가깝다.

group assignment보다는 *state/trajectory-conditioned sharing*에 가까움

환경 설정

- N 개의 agent가 같은 환경 안에서 동시에 행동한다.
- 각 agent i 는 자기 observation o_i^t 만 받는다.
- agent i 는 시점 t 에서 보이는 주변 agent 집합 $\mathcal{O}_i(t)$ 만 알 수 있다.
- 다른 agent의 reward function, critic parameter, 전체 observation은 직접 알 수 없다고 둔다.

관심 있는 상황

agent들이 완전히 동일하지 않고, role이나 reward/objective가 어느 정도 섞여 있는 환경

관측 가능한 정보

- 상대 agent의 내부 policy를 직접 비교하지 않는다.
- 대신 실제로 관측 가능한 움직임 정보를 사용한다.

정보	의미
상대 위치	나를 기준으로 상대 agent가 어디에 있는지
거리 변화	가까워지는지 멀어지는지
이동 방향	특정 목표나 object 쪽으로 움직이는지
관측 지속 시간	잠깐 보인 agent인지 계속 보이는 agent인지
action proxy	실제 action을 모를 때 움직임으로 근사한 행동 정보

Trajectory를 쓰는 이유

- 한 시점의 위치만 보면 상대 agent가 무엇을 보고 반응하는지 알기 어렵다.
- 몇 step 동안의 움직임을 보면 상대 agent의 의도나 관측 정보를 조금 더 추정할 수 있다.
- 예를 들어 prey를 본 predator는 방향을 바꾸거나 속도가 달라질 수 있다.
- 이런 움직임은 직접 관측하지 못한 정보를 간접적으로 전달할 수 있다.

핵심 가정

주변 agent의 trajectory에는 그 agent가 보고 있는 정보와 학습 방향이 일부 반영되어 있을 수 있다.

전체 구조

- 1 agent i 가 주변 agent j 의 최근 움직임을 관측한다.
- 2 짧은 trajectory를 encoder로 요약한다.
- 3 요약된 정보를 이용해 j 의 critic 정보가 얼마나 유용할지 점수화한다.
- 4 점수를 이용해 directed weighted graph를 만든다.
- 5 이 graph 위에서 critic 정보를 선택적으로 섞는다.

trajectory observation $\rightarrow$ utility score $\rightarrow$ directed critic sharing

Trajectory encoder

- agent i 가 본 agent j 의 최근 움직임을 짧은 sequence로 둔다.
- 이 sequence를 GRU나 LSTM으로 요약한다.
- 여기서 encoder는 상대 agent의 class를 맞추는 모델이 아니라, sharing 판단에 필요한 특징을 뽑는 역할이다.

$$h_{ij}^t = \text{Encoder}(\tau_{ij}^{t-k:t})$$

- h_{ij}^t 는 agent i 가 관측한 agent j 의 최근 행동 패턴을 요약한 값으로 본다.

Sharing score

- trajectory embedding을 이용해 agent i 가 agent j 를 얼마나 참고할지 score를 만든다.
- score는 “정책이 비슷한가”보다 “내 update에 도움이 될 것 같은가”에 가깝게 해석한다.
- 이 score를 normalize하면 critic sharing weight로 사용할 수 있다.

$$c_{ij}^t = \text{normalize}(\text{score}_{ij}^t)$$

중요한 점

c_{ij}^t 는 agent i 의 관점에서 만든 값이므로, 보통 c_{ij}^t 와 c_{ji}^t 는 같을 필요가 없다.

Critic sharing 방식

- 각 agent는 먼저 자기 transition으로 critic을 업데이트한다.
- 그 뒤 주변 agent의 critic update 또는 critic parameter를 weight에 따라 섞는다.
- 모든 agent의 정보를 같은 비율로 평균내지 않고, 현재 상황에서 유용하다고 판단한 agent를 더 크게 반영한다.

$$\omega_i^{t+1} \approx \text{weighted combination of local and neighbor critic updates}$$

- actor는 자기 observation을 기반으로 local하게 업데이트한다.
- 따라서 실행 단계에서 중앙 정보를 요구하지 않는다.

Directed graph로 보는 해석

- agent i 가 j 를 참고하는 것과 j 가 i 를 참고하는 것은 다른 문제이다.
- agent마다 보는 정보가 다르고, 필요한 정보도 다르기 때문이다.
- 그래서 공유 관계는 undirected edge보다 directed edge로 보는 것이 자연스럽다.

$i \rightarrow j$: “agent i 가 agent j 의 정보를 참고한다”

주의할 점

directed graph를 쓰면 단순 평균 consensus가 불안정할 수 있으므로, weight normalization이나 connectivity 조건을 같이 확인해야 한다.

알고리즘 흐름 초안

- ① 각 agent가 local observation으로 action을 선택한다.
- ② environment step 이후 local reward와 next observation을 받는다.
- ③ 관측된 주변 agent들의 최근 trajectory buffer를 업데이트한다.
- ④ trajectory encoder로 neighbor embedding을 계산한다.
- ⑤ neighbor별 sharing score를 계산하고 directed weight를 만든다.
- ⑥ local critic update를 계산한다.
- ⑦ weighted critic sharing을 수행한다.
- ⑧ local actor를 advantage 기반으로 업데이트한다.

현재 단계에서의 목표

우선은 이 흐름이 작은 환경에서 full sharing보다 안정적인지 확인하는 것이 먼저라고 생각한다.

실험에서 확인할 내용

- full sharing을 했을 때보다 학습이 안정적인가?
- no sharing보다 sample efficiency가 좋은가?
- fixed graph보다 환경 변화나 partial observation에 잘 대응하는가?
- learned directed edge가 실제 agent 역할이나 objective 차이와 관련이 있는가?
- 단순 trajectory similarity보다 learned sharing score가 나은가?

중요한 점

return만 보는 것이 아니라, 학습된 graph가 어떤 형태로 나오는지도 같이 봐야 한다.

방법	설명
Independent AC	agent마다 따로 actor-critic 학습
Full sharing	보이는 모든 이웃과 critic 정보를 공유
Fixed graph	미리 정한 neighbor graph 사용
Random graph	같은 edge 수를 random하게 선택
Trajectory similarity	가까운 trajectory를 가진 agent와 공유
Proposed	learned trajectory-conditioned directed weight 사용

- **Toy predator-prey**
 - reward/objective group을 명확히 설계할 수 있다.
 - graph가 제대로 분리되는지 확인하기 쉽다.
- **PettingZoo MPE**
 - simple spread 또는 simple tag를 변형해서 실험할 수 있다.
 - partial observation과 role 차이를 넣기 비교적 쉽다.
- **MAgent2 계열 환경**
 - agent 수를 늘리는 scaling 실험에 적합하다.
 - 다만 처음부터 쓰기에는 디버깅 비용이 클 수 있다.

- **Average return:** 기본 성능 비교
- **Learning speed:** 일정 성능에 도달하는 데 필요한 step 수
- **Stability:** seed 별 분산, collapse 여부
- **Communication sparsity:** 실제로 얼마나 적은 edge를 사용하는지
- **Graph pattern:** 같은 objective/role agent끼리 edge가 많이 생기는지
- **Directedness:** $i \rightarrow j$ 와 $j \rightarrow i$ 관계가 실제로 다르게 나오는지

- **Trajectory encoder 제거**: 현재 observation만으로 score 계산
- **Directed graph 제거**: weight를 symmetric하게 만들어 비교
- **Learned score 제거**: 단순 거리 또는 trajectory similarity 사용
- **Partial observation 강도 변화**: 관측 반경을 줄이거나 noise 추가
- **Communication budget 변화**: 사용할 수 있는 edge 수 제한

목적

성능 향상이 단순히 sharing을 더 많이 해서 나온 것인지, trajectory-conditioned directed weight 때문인지 분리해서 확인한다.

예상되는 어려움 1: score 학습 신호

- sharing score를 어떤 loss로 학습할지가 가장 애매한 부분이다.
- 단순히 return이 올라가도록 end-to-end로 학습하면 신호가 느리고 불안정할 수 있다.
- TD error 감소, advantage 변화, critic loss 변화 등을 보조 신호로 사용할 수 있을지 검토가 필요하다.

생각 중인 방향

처음에는 learned score와 단순 trajectory similarity를 모두 실험해 보고, 이후 TD-error 기반 utility를 붙이는 방향이 현실적일 것 같다.

예상되는 어려움 2: graph 안정성

- graph가 매 step 너무 크게 바뀌면 critic consensus 자체가 불안정해질 수 있다.
- directed graph에서는 정보가 특정 agent로만 몰리는 현상도 생길 수 있다.
- 따라서 weight smoothing, entropy regularization, minimum self-weight 같은 장치가 필요할 수 있다.

성능뿐 아니라 graph가 너무 흔들리지 않는지도 확인해야 함

예상되는 어려움 3: 차별점 정리

- communication learning 논문들은 이미 whom/when to communicate를 많이 다루고 있다.
- 그래서 이 주제를 단순 communication graph learning으로 설명하면 약해질 수 있다.
- 주된 차이는 action selection을 위한 message passing이 아니라, critic/update sharing을 위한 graph라는 점이다.
- 또한 SePS처럼 고정 group을 찾는 것이 아니라, 시점마다 바뀌는 directed relation을 다룬다는 점을 분명히 해야 한다.

현재 정리한 연구 방향

- 문제는 fully decentralized MARL에서 누구와 critic 정보를 공유할지 정하는 것이다.
- 입력으로는 각 agent가 실제로 볼 수 있는 주변 agent의 trajectory만 사용한다.
- 이 trajectory로부터 sharing score를 만들고, directed weighted graph를 구성한다.
- 그 graph를 actor-critic의 critic consensus 단계에 사용한다.

현재 버전의 핵심 문장

부분 관측 환경에서 주변 agent의 움직임을 이용해, critic sharing 관계를 동적으로 정하는 fully decentralized MARL 구조를 실험해 보고자 한다.

다음 단계

- ① 작은 toy environment를 먼저 설계한다.
- ② independent, full sharing, fixed graph baseline을 구현한다.
- ③ trajectory similarity 기반 sharing을 먼저 붙여본다.
- ④ 이후 GRU 기반 learned score로 확장한다.
- ⑤ 마지막으로 directed/undirected, partial observation 강도에 따른 ablation을 확인한다.

References

-  K. Zhang, Z. Yang, H. Liu, T. Zhang, and T. Basar, *Fully Decentralized Multi-Agent Reinforcement Learning with Networked Agents*, ICML, 2018.
-  F. Christianos, G. Papoudakis, A. Rahman, and S. V. Albrecht, *Scaling Multi-Agent Reinforcement Learning with Selective Parameter Sharing*, ICML, 2021.
-  P. Dai, W. Yu, H. Wang, and S. Baldi, *Distributed Actor-Critic Algorithms for Multiagent Reinforcement Learning Over Directed Graphs*, IEEE TNNLS, 2023.
-  A. Das et al., *TarMAC: Targeted Multi-Agent Communication*, ICML, 2019.
-  Y. Niu, R. Paleja, and M. Gombolay, *Multi-Agent Graph-Attention Communication and Teaming*, AAMAS, 2021.